Разработка модулей ядра Linux: Часть 5. Системные вызовы

Олег Цилюрик
преподаватель тренингового отделения
Global Logic

03.05.2012

В этой статье описываются схемы взаимодействия между пользовательскими процессами и модулями или службами ядра и различные схемы взаимодействия между модулями и ядром.

Введение

Чтобы полностью разобраться, чем является модуль для ядра, необходимо вернуться назад и рассмотреть следующие вопросы:

- как пользовательские процессы взаимодействуют с сервисами ядра;
- что представляют собой системные вызовы;
- какие интерфейсы существуют между пользователем и ядром; пользователем и модулем, как составной частью ядра; и между модулем и ядром.

Траектория системного вызова

Для полного описания взаимодействия между всеми вышеперечисленными участниками недостаёт ещё одного фрагмента: механизма взаимодействия **пользовательских процессов** с ядром и модулем. В конечном итоге, главная задача операционной системы — это обслуживание потребностей прикладных пользовательских процессов. И обеспечивается это обслуживание через механизм **системных вызовов**. Если сказать, что операционная система 99.999% своего рабочего времени занимается обслуживанием системных вызовов, и только на оставшуюся часть приходится все остальные активности, как обработка прерываний, обслуживание таймеров, диспетчеризация потоков и другие подобные «мелочи», то это не будет слишком большим преувеличением! Для данного утверждения есть стандартное возражение, что «для серверов это не верно». Возможно, но если учесть, что **вся** деятельность серверов связана с обслуживанием всё тех же системных вызовов, но запрошенных с других узлов сети, то это полностью соответствует представленному утверждению. Поэтому изучение вопросов взаимосвязей и взаимодействий в операционной

© Copyright IBM Corporation 2012

системе всегда нужно начинать с рассмотрения цепочки, по которой проходит системный вызов.

В **любой** (в том числе и **микроядерной**) операционной системе системный вызов выполняется некоторой выделенной процессорной инструкцией, прерывающей последовательное выполнение команд и передающий управление коду режима супервизора. Это обычно некая команда **программного** прерывания, в зависимости от архитектуры процессора в разные времена это были команды с мнемониками вида: `svc`, `emt`, `trap`, `int` и им подобными. Если обратиться только к архитектуре Intel x86, то в ней для этого традиционно используется команда программного прерывания с различным вектором. В таблице 1 приведены команды прерываний для различных операционных систем.

Таблица 1. Команды прерывания для ОС, построенных на архитектуре Intel x86

Операционная система	Дескриптор прерывания для системного вызова
MS-DOS	21h
Windows	2Eh
Linux	80h
QNX	21h
MINIX 3	21h

Примечание: Начиная с определенного момента (примерно с начала 2008 года или момента выхода Windows XP Service Pack 2) многие операционные системы (Windows, Linux) отказались от использования программного прерывания `int`, и перешли к реализации системного вызова и возврата из него через новые команды процессора `sysenter` (`sysexit`). Это было связано с заметным падением производительности Pentium IV при классическом способе системного вызова и желанием восстановить эту производительность любой ценой. Но ничего принципиально нового в данном случае не появилось, так как ключевые параметры перехода (`CS`, `SP`, `IP`) просто стали загружаться не из памяти, а из специальных внутренних регистров `MSR` (Model Specific Registers) с предопределёнными (0x174, 0x175, 0x176) номерами (из большого их общего числа), куда предварительно эти значения записываются специальной новой командой процессора `wmsr`. По описанию можно заметить, как это громоздко, но производительность выросла, а по сути: «вектор прерывания теперь записывается в аппаратном обеспечении, а процессор помогает быстрее перейти с одного уровня привилегий на другой».

Библиотечный и системный вызов из процесса

Теперь можно переходить к детальному рассмотрению трассировки системного вызова в Linux (рассмотрение основано на классической реализации через команды `int 80h / iret`, так как реализация через `sysenter / sysexit` ничего принципиально нового не вносит). Прикладной процесс вызывает требуемые ему службы посредством библиотечного вызова к множеству библиотек либо вида ***.so** (динамическое связывание), либо прикомпоновывая к себе фрагмент из библиотеки вида ***.a** (статическое связывание). Самые известные примеры

- это стандартная библиотека языка C **libc.so** или **libpthread.so** — библиотека POSIX потоков. Значительная часть вызовов обслуживается непосредственно внутри библиотеки, не требуя никакого вмешательства со стороны ядра, например, вызов `sprintf()` или все строковые POSIX-функции вида `str*()`. Другая часть требует дальнейшего обслуживания со стороны ядра, например, вызов `printf()` (синтаксически близкий к `sprintf()`). Тем не менее, **все** подобные вызовы API классифицируются как **библиотечные вызовы**. В Linux чётко регламентируются группы вызовов, при этом библиотечные API относятся к секции 3 в руководствах `man`. Хорошим примером может служить целая группа функций для запуска дочернего процесса `execl()`, `execlp()`, `execle()`, `execv()`, `execvp()`:

```
$ man 3 exec
NAME
       execl, execlp, execle, execv, execvp - execute a file
```

Хотя ни один из этих **библиотечных** вызовов не запускает непосредственно дочерний процесс, а только ретранслируют вызов единственному **системному** вызову `execve()`:

```
$ man 2 execve
...
```

Описания **системных** вызовов (в отличие от библиотечных) отнесены к секции 2 в руководствах `man`. Все системные вызовы далее преобразуются в вызов ядра функцией `syscall()`, 1-м параметром которого будет идентификатор выполняемого системного вызова, например `__NR_execve`. Таким образом, справочную информацию по библиотечным функциям необходимо искать в секции 3, а по системным вызовам — в секции 2:

```
$ man 3 printf
...
$ man 2 write
...
```

Ещё один простой пример, поясняющий данный аспект:

- библиотечный вызов `printf( "%s", string )` всегда вызывает **библиотечную** функцию `sprintf()` для формирования выводимой строки, которая в данном простом случае будет совпадать со `string`;
- далее из `printf()` выполняется **системный** вызов `write( 1, string, strlen( string ) )` ;
- который трансформируется в вызов `sys_call( __NR_write, ...)` ;
- который затем выполнит команду `int 0x80` (полный код такого примера будет рассмотрен ниже);
- и в конце управление будет передано **обработчику системного вызова** в пространстве ядра.

Информацию о вызове `syscall()` можно посмотреть в справочнике man:

```
$ man syscall
ИМЯ
     syscall - непрямой системный вызов
...
```

Образцы численных идентификаторов системных вызовов из верхней части таблицы уже рассматривались ранее.

Листинг 1. Идентификаторы системных вызовов

```
$ head -n6 /usr/include/asm/unistd_32.h
#define __NR_exit                 1
#define __NR_fork                 2
#define __NR_read                 3
#define __NR_write                4
#define __NR_open                 5
#define __NR_close                6
```

Кроме `syscall()` в Linux поддерживается и другой механизм системного вызова — `lcall7()`, в котором устанавливается шлюз системного вызова для поддержки стандарта iBCS2 (Intel Binary Compatibility Specification), благодаря чему на x86 Linux может выполняться **бинарный код**, подготовленный для других операционных систем: FreeBSD, Solaris/86, SCO Unix. Но этот механизм в рамках данного цикла статей рассматриваться не будет.

Системные вызовы `syscall()` в Linux **на процессоре x86** выполняются через прерывание `int 0x80`. Соглашение о системных вызовах в Linux отличается от общепринятого в UNIX и соответствует типу «fastcall». Согласно ему, программа помещает в регистр `eax` идентификатор системного вызова, входные аргументы размещаются в других регистрах процессора (таким образом, системному вызову может быть передано до 6 аргументов через регистры в такой последовательности: `ebx, ecx, edx, esi, edi` и `ebp`), после чего вызывается инструкция `int 0x80`. В тех редких случаях, когда системному вызову необходимо передать **большее количество** аргументов, то они размещаются в структуре, адрес на которую передается в качестве первого аргумента (`ebx`). Результат возвращается в регистре `eax`, а стек вообще не используется. Системный вызов `syscall()`, попав в ядро, всегда попадает в таблицу `sys_call_table`, и далее переадресовывается по индексу (смещению) в этой таблице на величину 1-го параметра вызова `syscall()` - идентификатора требуемого системного вызова.

Заключение

Подробный пример реализации системного вызова будет рассмотрен в одной из следующих статей, где он поможет изучить отличия между пользовательским и ядерным программированием.

Об авторе

Олег Цилюрик



Олег Иванович Цилюрик, много лет был разработчиком программного обеспечения в крупных центрах разработки: ВНИИ РТ, НПО "Дельта", КБ ПМ. Последние годы работал над проектами в области промышленной автоматики, IP телефонии и коммуникаций. Автор нескольких книг. Преподаватель тренингового отделения международной софтверной компании Global Logic.

© Copyright IBM Corporation 2012
(www.ibm.com/legal/copytrade.shtml)
Торговые марки
(www.ibm.com/developerworks/ru/ibm/trademarks/)